Data 100, Spring 2026

Discussion #12 *Solutions*

Note: Your TA will probably not cover all the problems. This is totally fine; the discussion worksheets are not designed to be finished within an hour. They are deliberately made slightly longer so they can serve as resources you can use to practice, reinforce, and build upon concepts discussed in lectures, labs, and homework.

Learning Objectives

Logistic Regression

Regression vs Classification: Distinguish between regression and classification problems and identify when logistic or linear regression is appropriate.

Logistic Model and Sigmoid: Write the logistic regression model and the sigmoid function. Describe the shape of the sigmoid function. Describe how the sigmoid transforms a linear combination of features and parameters into an estimated probability.

Cross-Entropy Loss: Define cross-entropy (CE) loss for logistic regression. Explain why we use CE loss instead of MSE/MAE.

Decision Rules and Linear Separability: Explain how probabilities from logistic regression are converted into predicted classes using a threshold. Explain how to extract the equation of a linear decision boundary from a fitted logistic regression. Define *linearly separable* in the context of classification.

Performance Metrics: Define accuracy. Explain why accuracy can be misleading. Define true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN). Describe a confusion matrix. Define precision, recall, TPR, and FPR. Describe a precision-recall curve. Describe how to construct an ROC curve from a set of estimated probabilities and true outcomes.

Relevant Reference Sheet Excerpts

Logistic Regression and Classification

Logistic Regression Model: For input feature vector x, $\hat{P}_\theta(Y=1|x) = \sigma(x^T\theta)$, where $\sigma(z) = 1/(1+e^{-z})$. For a single datapoint, define cross-entropy loss as $-[y\log(p) + (1-y)\log(1-p)]$, where p is the probability that the response is 1.

An ROC curve has the false positive rate (FPR) on the x-axis and true positive rate (TPR) on the y-axis.

Classification Performance

Suppose you predict n datapoints.

Metric	Formula
Accuracy	$\frac{TP+TN}{n}$
Precision	$\frac{TP}{TP+FP}$
Recall, True Positive Rate (TPR)	$\frac{TP}{TP+FN}$
False Positive Rate (FPR)	$\frac{FP}{FP+TN}$
F1 Score	$F1 = 2 \cdot \frac{\text{Precision}\cdot\text{Recall}}{\text{Precision}+\text{Recall}}$

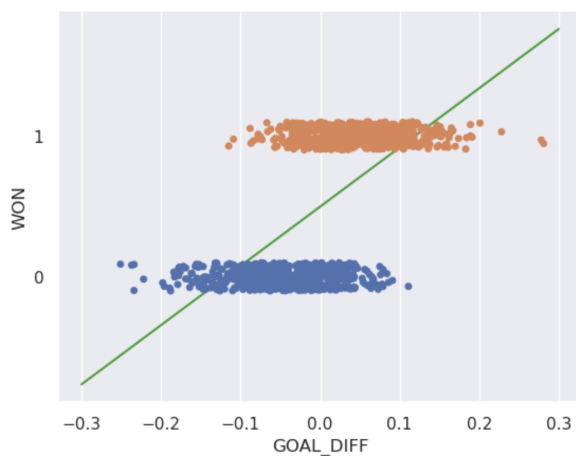
Deriving Logistic Regression

Odds is the ratio of the probability of y being Class 1 to the probability of y being Class 0.

$$odds = \frac{P(y=1|x)}{P(y=0|x)} = \frac{p}{1-p}$$

1. In this question, we'll walk through a transformation from the Linear Regression equation to the Logistic Regression equation.

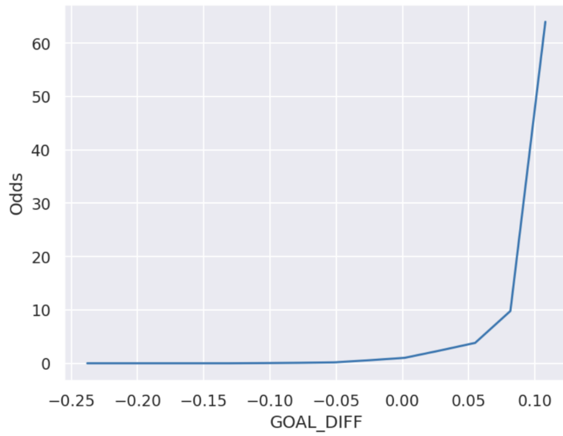
   The goal of Linear Regression is to predict y given an input x, and this is accomplished by learning the parameter θ. While this model can be useful when predicting things like house prices in Project A, it does less well when predicting *binary* variables of 0 or 1.



   (a) How can we modify our Linear Regression equation, $y = x^T\theta$, to fit this new type of problem? Instead of predicting the value of y, which can only be 0 or 1, let's predict the *probability* p that $y = 1$. Under this model, what's the probability that $y = 0$? What is the range of values that p could take?

   Solution: Since $\Pr(y = 1) = p$, and Y can only take on two values, 0 and 1, $\Pr(y = 0)$ must be equal $1 - p$.
   Note: $0 \leq p \leq 1$ because p is a probability.

(b) Our goal is to modify our original function, $x^T\theta$, so that it's range of values is [0, 1] instead of $(-\infty, +\infty)$. To do this, we'll perform a series of transformations on $x^T\theta$ starting with *odds*.



What is the range of *odds*?

- ○ A. [-1, 1]
- ○ B. $(0, +\infty)$
- ○ C. $[0, +\infty)$
- ○ D. $(-\infty, +\infty)$

Solution: The range of p is constrained from 0 to 1.

$$\lim_{p \to 0} \frac{p}{1-p} = 0$$

$$\lim_{p \to 1} \frac{p}{1-p} = +\infty$$

(c) In logistic regression, we use **log-odds**:

$$log(odds) = log\left(\frac{p}{1-p}\right)$$

What is the range of $log(odds)$? Does it match the range of our Linear Regression model?

- ○ A. [0, 1]
- ○ B. [-1, 1]
- ○ C. $[0, \infty)$
- ○ D. $(-\infty, \infty)$

Solution: Yes, the log-odds range and the Linear Regression model prediction $x^T\theta$ range is the same.

(d) Putting this all together, we have:

$$log(\frac{p}{1-p}) = x^T\theta$$

Rearranging the terms, we can solve for p:

$$p = \frac{1}{1 + e^{-x^T\theta}} = \sigma(x^T\theta)$$

and obtain our logistic function σ. What is the range of values that σ could take?

- ○ A. [0, 1]
- ○ B. (0, 1)
- ○ C. [-1, 1]
- ○ D. $[0, \infty)$
- ○ E. $(-\infty, \infty)$

Logistic Regression

In logistic regression, we model the *probability* that $Y = 1$ given input features $\vec{x}$, rather than predicting Y directly. We denote probability as $P_{\hat{\theta}}$ as opposed to just P to show that $\hat{\theta}$ is a *learned parameter* like it is in OLS, where we denote our function as $f_\theta(x)$.

$$P_{\hat{\theta}}(Y = 1|\vec{x}) = \sigma(\vec{x}^T\theta) = \frac{1}{1 + \exp(-\vec{x}^T\theta)}$$

$\sigma(z)$ is called a "sigmoid" function with the equation

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

for some arbitrary real number z. The range of possible values for $\sigma(\cdot)$ is $(0, 1)$.
Logistic regression provides a *linear decision boundary.*
The empirical risk using **cross-entropy loss** is given by the following expression. Remember, whenever you see log in this course, you must assume the natural logarithm (base-e) unless explicitly told otherwise.

$$R(\theta) = -\frac{1}{n}\sum_{i=1}^{n}\big(y_i \log P_\theta(Y = 1|\vec{x_i}) + (1 - y_i)\log P_\theta(Y = 0|\vec{x_i})\big)$$

2. Suppose we are given the following dataset, with two features ($\mathbb{X}_{:,0}$ and $\mathbb{X}_{:,1}$) and one binary response variable (y).

$\mathbb{X}_{:,0}$	$\mathbb{X}_{:,1}$	y
2	2	0
1	-1	1

Here, $\vec{x}^T$ corresponds to a single row of our data matrix, not including the y column. Thus, we can write $\vec{x}_1^T$ as $\vec{x}_1^T = [2 \quad 2]$. Note that there is no intercept term!

Suppose you run a Logistic Regression model to determine the probability that $Y = 1$ given $\vec{x}$.

Your algorithm learns that the optimal $\hat{\theta}$ value is $\hat{\theta} = \left[-\frac{1}{2} \quad -\frac{1}{2}\right]^T$.

(a) Calculate $P_{\hat{\theta}}(Y = 1|\vec{x}^T = \begin{bmatrix}1 & 0\end{bmatrix})$.

Solution:

$$\begin{aligned} P_{\hat{\theta}}(Y = 1|\vec{x}^T = [1 \quad 0]) &= \sigma\left(\begin{bmatrix}1 & 0\end{bmatrix}\begin{bmatrix}-\frac{1}{2} \\ -\frac{1}{2}\end{bmatrix}\right) \\ &= \sigma\left(1 \cdot -\frac{1}{2} + 0 \cdot -\frac{1}{2}\right) \\ &= \sigma\left(-\frac{1}{2}\right) \\ &= \frac{1}{1 + \exp(\frac{1}{2})} \\ &\approx 0.38 \end{aligned}$$

(b) Using a threshold of $T = 0.5$, what would our algorithm classify y as given the results of part (b)?

Solution: Our p of 0.38 is smaller than the threshold of 0.5, so our algorithm would classify y as class 0.

(c) Suppose we run a different algorithm and obtain $\hat{\theta}_{new} = [0 \quad 0]^T$. Calculate the empirical risk for $\hat{\theta}_{new}$ on our dataset.

Solution:

$$
\begin{aligned}
R(\hat{\theta}_{new}) &= -\frac{1}{2}\sum_{i=1}^{2}\left(y_i \log P_{\hat{\theta_{new}}}(Y=1|\vec{x_i}) + (1-y_i)\log P_{\hat{\theta_{new}}}(Y=0|\vec{x_i})\right) \\
&= -\frac{1}{2}[(0\log P_{\hat{\theta_{new}}}(Y=1|\vec{x_1}) + 1\log P_{\hat{\theta_{new}}}(Y=0|\vec{x_1}))+ \\
&(1\log P_{\hat{\theta_{new}}}(Y=1|\vec{x_2}) + 0\log P_{\hat{\theta_{new}}}(Y=0|\vec{x_2}))] \\
&= -\frac{1}{2}(\log P_{\hat{\theta_{new}}}(Y=0|\vec{x_1}) + \log P_{\hat{\theta_{new}}}(Y=1|\vec{x_2})) \\
&= -\frac{1}{2}(\log(1-\sigma\left(\begin{bmatrix}2 & 2\end{bmatrix}\begin{bmatrix}0\\0\end{bmatrix}\right)) + \log\sigma\left(\begin{bmatrix}1 & -1\end{bmatrix}\begin{bmatrix}0\\0\end{bmatrix}\right)) \\
&= -\frac{1}{2}(\log(1-\sigma(0)) + \log\sigma(0)) \\
&= -\log(0.5) = \log 2 \approx 0.693
\end{aligned}
$$

3. Suppose we have two different Logistic Regression models, A and B, and we run gradient descent for 1000 steps to obtain the model parameters $\hat{\theta}_A = \begin{bmatrix} -\frac{1}{2} & -\frac{1}{2} \end{bmatrix}^T$ and $\hat{\theta}_B = \begin{bmatrix} 0 & 0 \end{bmatrix}^T$. How do they compare?

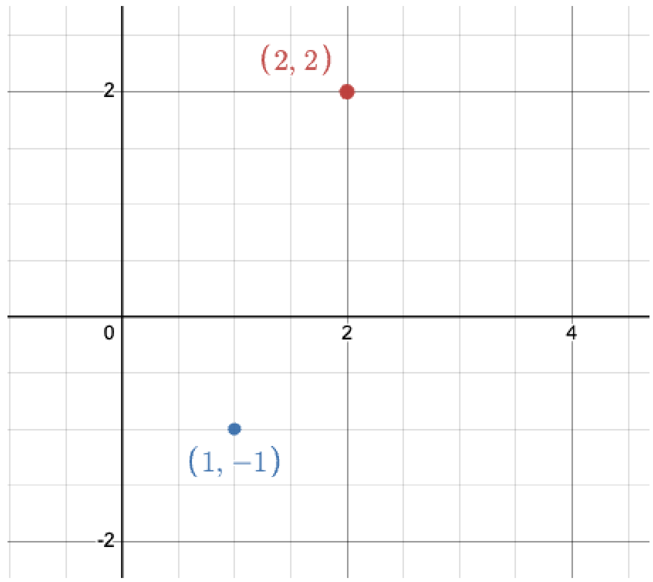
   The dataset is reproduced below for your convenience.

$\mathbb{X}_{:,0}$	$\mathbb{X}_{:,1}$	y
2	2	0
1	-1	1

   (a) Is our dataset linearly separable? If so, write the equation of a hyperplane that separates the two classes. Otherwise, briefly explain why not.

   Hint: Draw the two data points.

   Solution: Yes, the line $\mathbb{X}_{:,1} = 0$ separates the data in feature space.



   Note: The horizontal axis is $\mathbb{X}_{:,0}$ and the vertical axis is $\mathbb{X}_{:,1}$.

   (b) If we let gradient descent keep running indefinitely for our two models, will either of them converge given the design matrix above? Why? If not, how can we remedy this?

   Solution: No.

   Our dataset is linearly separable, so the optimal cross-entropy loss is 0. However, a cross-entropy loss of 0 can never be achieved. Remember that $\sigma(z)$ never outputs precisely 0 or 1, but it can get arbitrarily close. Hence, no single value of θ will ever "minimize" cross-entropy loss (ie. let $\sigma(x^T\theta) = 0$)), but gradient descent can bring the cross-entropy loss closer and closer to 0 as θ goes to $\pm\infty$.

   To avoid our absolute values of the weights diverging to ∞, we can regularize our cross-entropy loss.

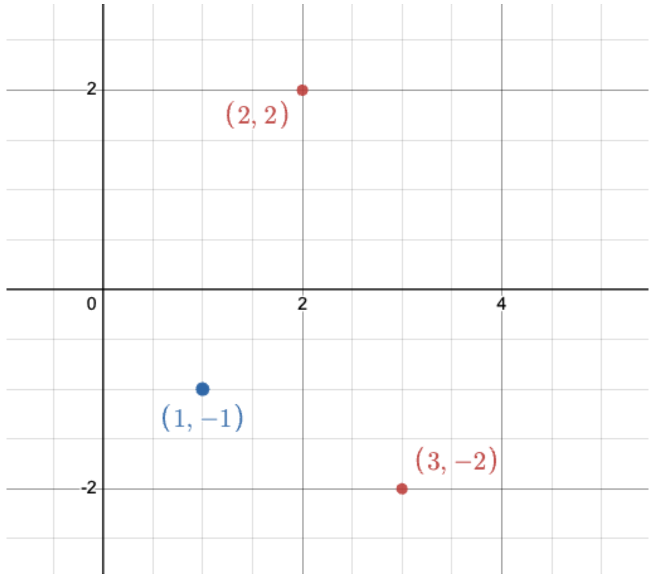
   Note: In the real world, since computers have finite precision (decimals can only go so far) it will eventually converge. But without this limitation, in the ideal world gradient descent will never converge.

   (c) Assume we add the data point $[3, -2]$ to the design matrix such that our resulting design matrix is as follows.

$\mathbb{X}_{:,0}$	$\mathbb{X}_{:,1}$	y
2	2	0
1	-1	1
3	-2	0

   Is it possible to achieve perfect accuracy using a logistic regression model?

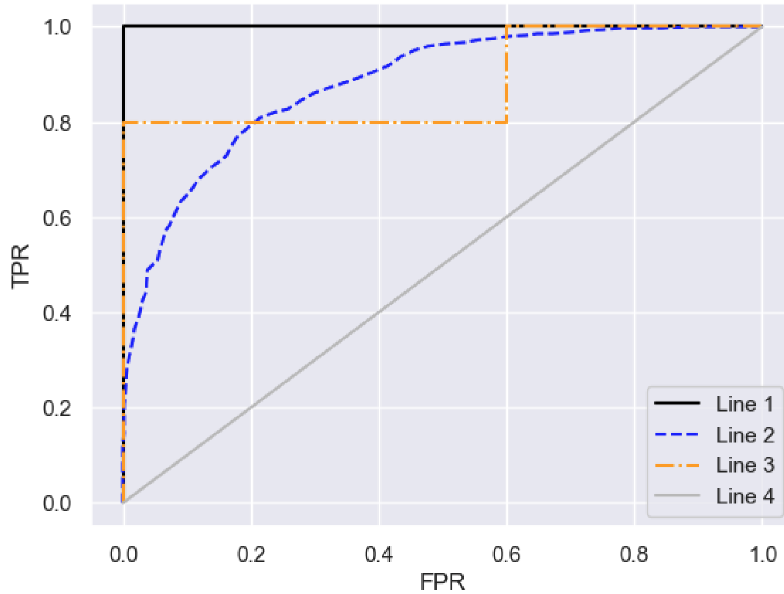
Solution: The data is still linearly separable, so we can train a logistic regression model to achieve perfect accuracy!



Note: The horizontal axis is $\mathbb{X}_{:,0}$ and the vertical axis is $\mathbb{X}_{:,1}$.

ROC Curves

4. Consider the following ROC (receiver operating characteristic) curves that were each created from different models.



(a) Compare the Area Under the Curve (AUC) of Line 1 and Line 4. What kind of model would predict Line 1? What kind of model would predict Line 4?

Solution: Line 1 (solid black line) is known as a *perfect predictor*, meaning for this model there exists a threshold that perfectly predicts the correct class for all y. At this threshold value the true positive rate (TPR) is 1, and its false positive rate (FPR) is 0. Because we want our classifier to be as close as possible to the perfect predictor, we aim to maximize the AUC.

On the other hand, Line 4 (solid grey line) is a *random predictor* that predicts $y = 1$ with a probability of 0.5 and $y = 0$ with a probability of 0.5. Its AUC = 0.5, and it does no better than a random coin flip.

Note: While we aim to maximize AUC, an AUC under 0.5 can still be informative. We can invert this classifier (i.e. predicting 0 instead of 1 and vice versa by taking the complement of each predicted probability).

(b) Suppose we fix the decision threshold for all 4 models such that we get an FPR of 0.1 when we evaluate our model. In order of most to least preferred, rank models given their ROC curve.

Solution: Since the FPR is now fixed at 0.1, the only thing that we can adjust is the TPR. To determine the most to least preferred model, we look at each model's TPR at an FPR of 0.1. Higher TPRs are better because they indicate that a greater proportion of true positive cases are correctly identified by the model. Hence, we get the following ranking:

$$\text{Line 1} > \text{Line 3} > \text{Line 2} > \text{Line 4}$$

Performance Metrics

5. Suppose we train a binary classifier on the following dataset where y is the set of true labels, and $\hat{y}$ is the set of predicted labels:

y	0	0	0	0	0	1	1	1	1	1
$\hat{y}$	0	1	1	1	1	1	1	0	0	0

(a) Fill out the confusion matrix for the given data.

Hint: The first row contains the true negatives and false positives, and the second row contains false negatives and true positives (in that order).

TN:	FP:
FN:	TP:

Solution: $\begin{bmatrix} \text{TN: } 1 & \text{FP: } 4 \\ \text{FN: } 3 & \text{TP: } 2 \end{bmatrix}$

(b) The precision of our classifier. Write your answer as a simplified fraction.

Solution: $\frac{2}{2+4} = \frac{1}{3}$

(c) The recall of our classifier. Write your answer as a simplified fraction.

Solution: $\frac{2}{2+3} = \frac{2}{5}$

(d) It is revealed that this dataset describes the results of an algorithm used to predict whether someone is at risk of developing a severe disease with expensive treatment. You are tasked with improving the classifier. Which metrics should you aim to optimize for (Accuracy/Precision/Recall)? Explain your reasoning.

Hint: Things to consider: cost of treatment, the severity of disease.

Solution: There is no singular correct answer, but here are some examples of reasons.

Accuracy: Since this dataset is fairly balanced, accuracy is not too bad of a metric, which is equal to $\frac{3}{10}$ here. The main flaw of using accuracy is that it treats all errors the same regardless of which class a data point belongs to. This means it fails to show whether the model is performing worse on certain classes, especially if a class is rare.

Precision: Optimizing for precision means that we care more about making sure the positives we output are truly positive. In this setting, we want to ensure that the people we predict to have the disease truly have the disease. If the cost of treatment is very expensive, we don't want to overburden people who may not have this disease financially.

Recall: Optimizing for recall means we care more about detecting all the true positives from the dataset. In this setting, we want to make sure that almost everyone who has the disease knows they have it. If the disease is particularly deadly, then we would be aiming to save the most lives.